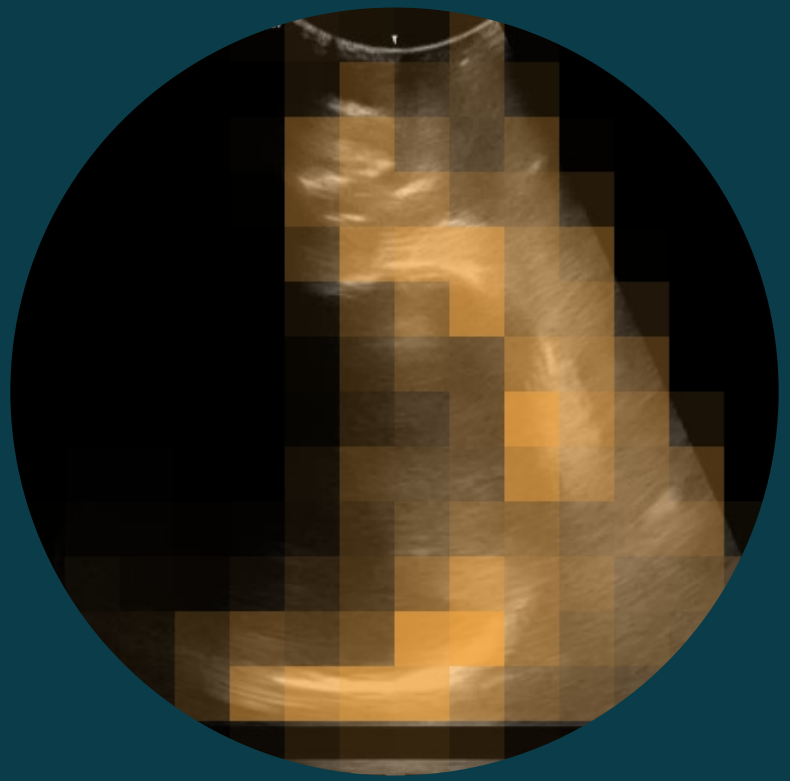
Learning where to mask

Reinforcement-learned masking policies for V-JEPA pretraining on intrapartum ultrasound video



Motion map of one training clip

Project summary

The method runs end to end; its benefit is not yet shown

What we built

- A masking policy that learns where V-JEPA should hide video patches, trained with reinforcement learning
- Two versions: REINFORCE places mask blocks; PPO with trajectory attention picks the visible tokens

What we observed

- All pretraining runs completed; PPO needed stability fixes first
- Both policies stayed close to random masking
- PPO token masking ran in about 17 GB, against 35 to 45 GB for block masking

Where it stands

- Frame classification: no pretrained model clearly beats an untrained encoder
- Pretraining loss does not track feature quality
- The evaluation setup is the next thing to fix

Status: a working, stable implementation and an evaluation pipeline; evidence of better features is still missing.

1

Background

The data, how V-JEPA learns, and how it normally chooses its masks

The data: intrapartum ultrasound videos with plane labels



One frame of a training video

434

training videos
266 SP, 168 NSP

40

validation videos
2,870 labelled frames

300

test videos
8,665 labelled frames

Task

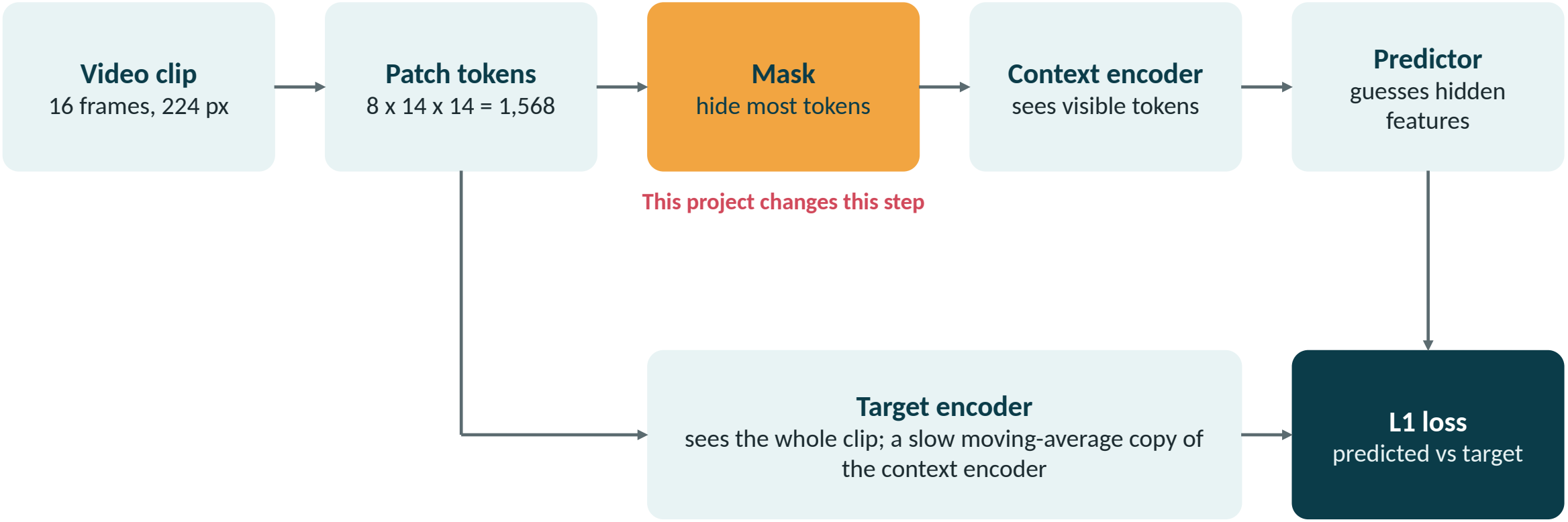
Classify each frame as standard plane (SP) or non-standard plane (NSP)

Labels

Training: one label per video. Validation and test: one label per frame, with both classes inside each video

512 x 512 greyscale at 24 fps. Training videos have 9 to 778 frames; test videos 20 to 70. Source: IUGC 2024 dataset (MICCAI).

V-JEPA learns by predicting hidden parts of a video as features



No pixels are reconstructed. Gradients update the context encoder and the predictor; the target encoder follows as a moving average.

Standard V-JEPA places its mask blocks at random

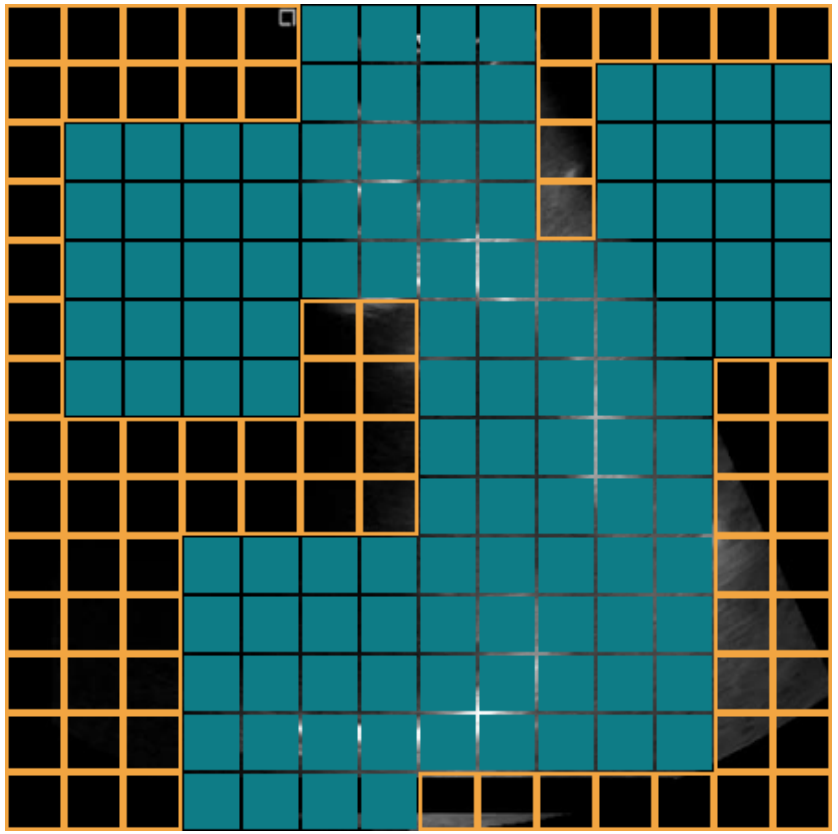


Illustration: teal patches are hidden, outlined patches stay visible

Random multi-block masks

One mask uses 8 small blocks (15% of the frame each); a second uses 2 large blocks (70% each)

Blocks span the whole clip

Each block covers all 8 time slices, so the model cannot copy the answer from another frame

Placement ignores content

Block positions are uniformly random, by the same rule for every video

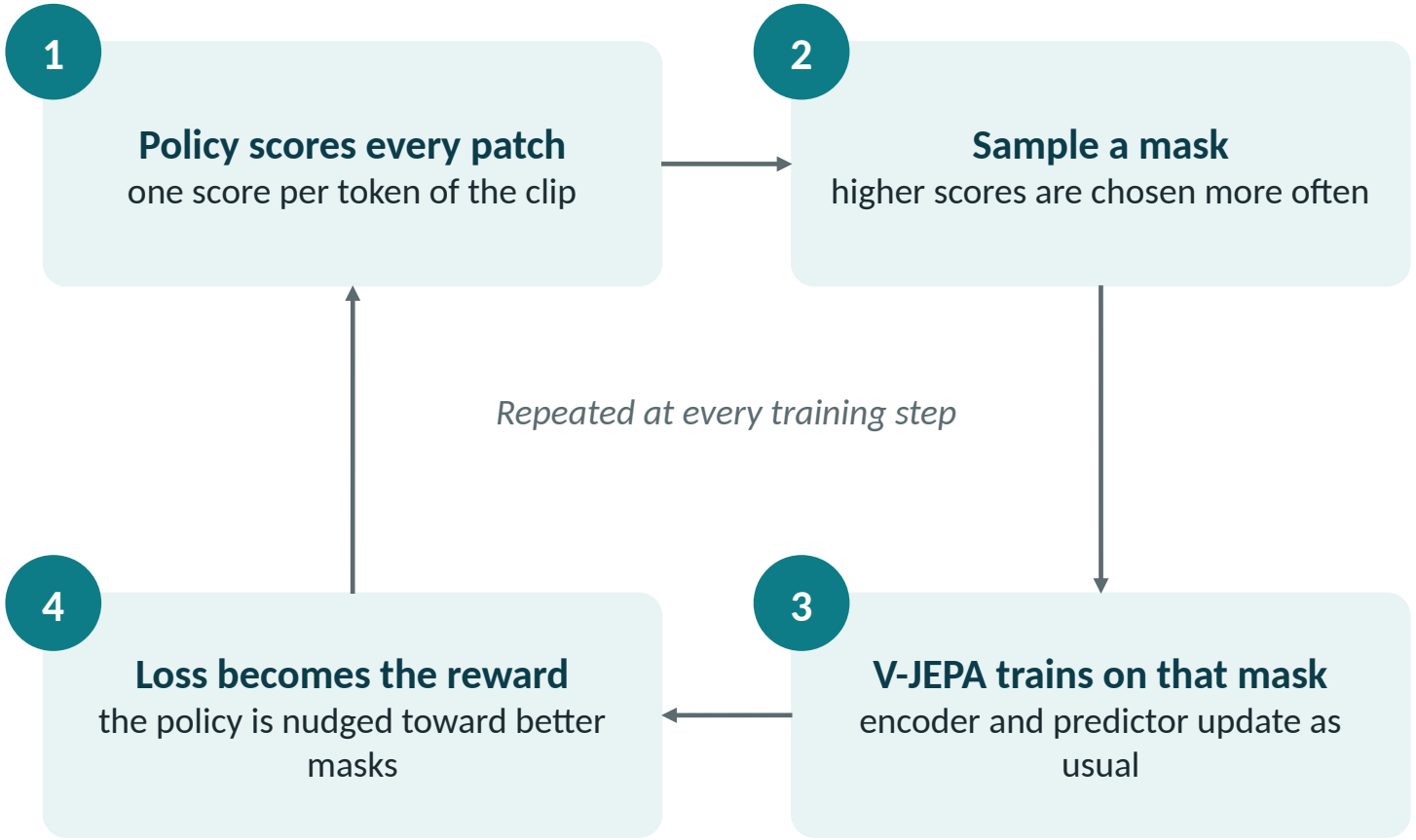
Question: can a model learn where masking is most useful for these videos?

2

Our modification

A masking policy trained with reinforcement learning, in two versions

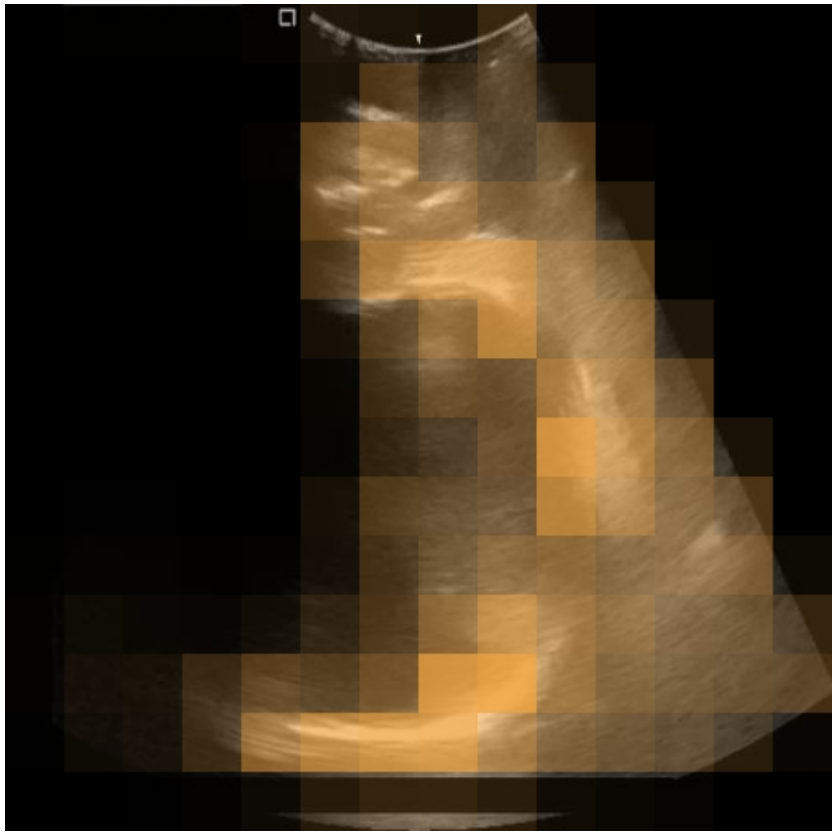
A policy chooses the mask and learns from V-JEPA's loss



Why reinforcement learning?

- Choosing a mask is a discrete decision, so the V-JEPA loss cannot be back-propagated into the policy
- The policy therefore learns from rewards computed from that loss
- Two versions were built: REINFORCE and PPO

Version 1: REINFORCE decides where the mask blocks go



Real motion map of a training clip: brighter patches change more between frames

1. **Score every patch**
   A small transformer score, plus a learned weight times the motion map

2. **Place the blocks**
   Block centres are sampled from the scores; block sizes stay random as in V-JEPA

3. **Reward**
   Minus the clip's prediction loss, so easier masks earn more

4. **Update**
   REINFORCE with a moving-average baseline and an entropy bonus

Version 2: PPO picks which tokens stay visible

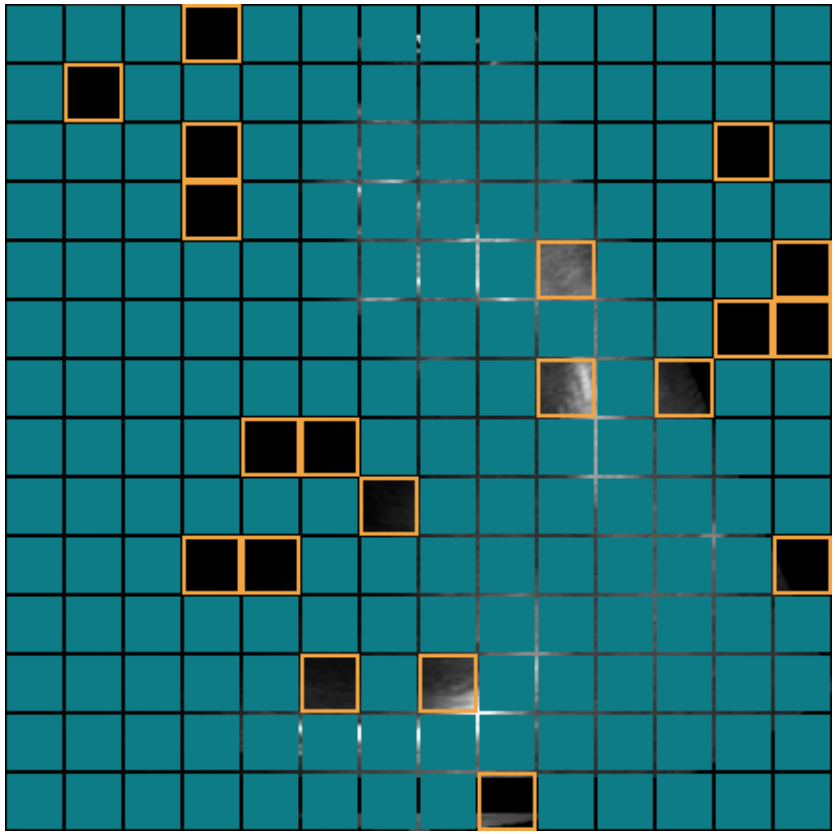


Illustration: 10% of patches stay visible (outlined); the other 90% are predicted

1. **Input** The encoder's own patch embeddings of the clip
2. **Actor** A trajectory-attention block follows each patch through time and scores it
3. **Action** Sample 156 of 1,568 tokens to keep visible; the other 1,412 become targets
4. **Reward** Each target token's prediction loss, so harder tokens earn more
5. **Update** PPO with a critic network; the first 10 epochs use random masks

Adapted from TATS (Rai et al., ICCV Workshops 2025), which was designed for VideoMAE pixel reconstruction.

The versions differ in what they choose and reward

	REINFORCE	PPO (TATS-style)
Chooses	Where the rectangular mask blocks go	Which individual tokens stay visible
Looks at	Small transformer plus a frame-difference motion map	Trajectory attention over encoder embeddings
Reward	Lower prediction loss: favours easy masks	Higher per-token loss: favours hard masks
Learning rule	REINFORCE with a moving-average baseline	PPO: actor, critic and clipped updates
Visible context	About 270 and 55 tokens in the two masks	156 tokens (10%) in one mask
Origin	Designed in this project	Ported from TATS (VideoMAE) to V-JEPA

The two rewards point in opposite directions, so a final method has to commit to one hypothesis.

The first PPO run crashed; four fixes made it stable

First run, epoch 11

5.6 to 0.1

policy entropy within about 20 steps

2.7×10^{15}

policy loss just before the crash

Scores became NaN and mask sampling failed.

1 Normalised policy input

Encoder embeddings grew large and made the untrained policy almost deterministic. A LayerNorm removed this root cause.

2 Probability floor

Every token keeps a small chance of being picked, so log-probabilities stay bounded.

3 Bounded PPO loss

Clamped ratio, dual-clip for negative advantages, and updates with non-finite gradients are skipped.

4 Repeatable scoring

Attention landmarks are chosen deterministically, so the update sees the same scores as sampling.

Result: 290 PPO epochs completed with no skipped updates and no low-entropy warnings.

3

Pretraining results

Cost, loss curves, and what the two policies actually learned

All runs share one training setup; only the masking changes

ViT-L/16
encoder

16 x 224 px
frames per clip

24
clips per batch

300 x 300
epochs x steps

1 x A100
GPU per run

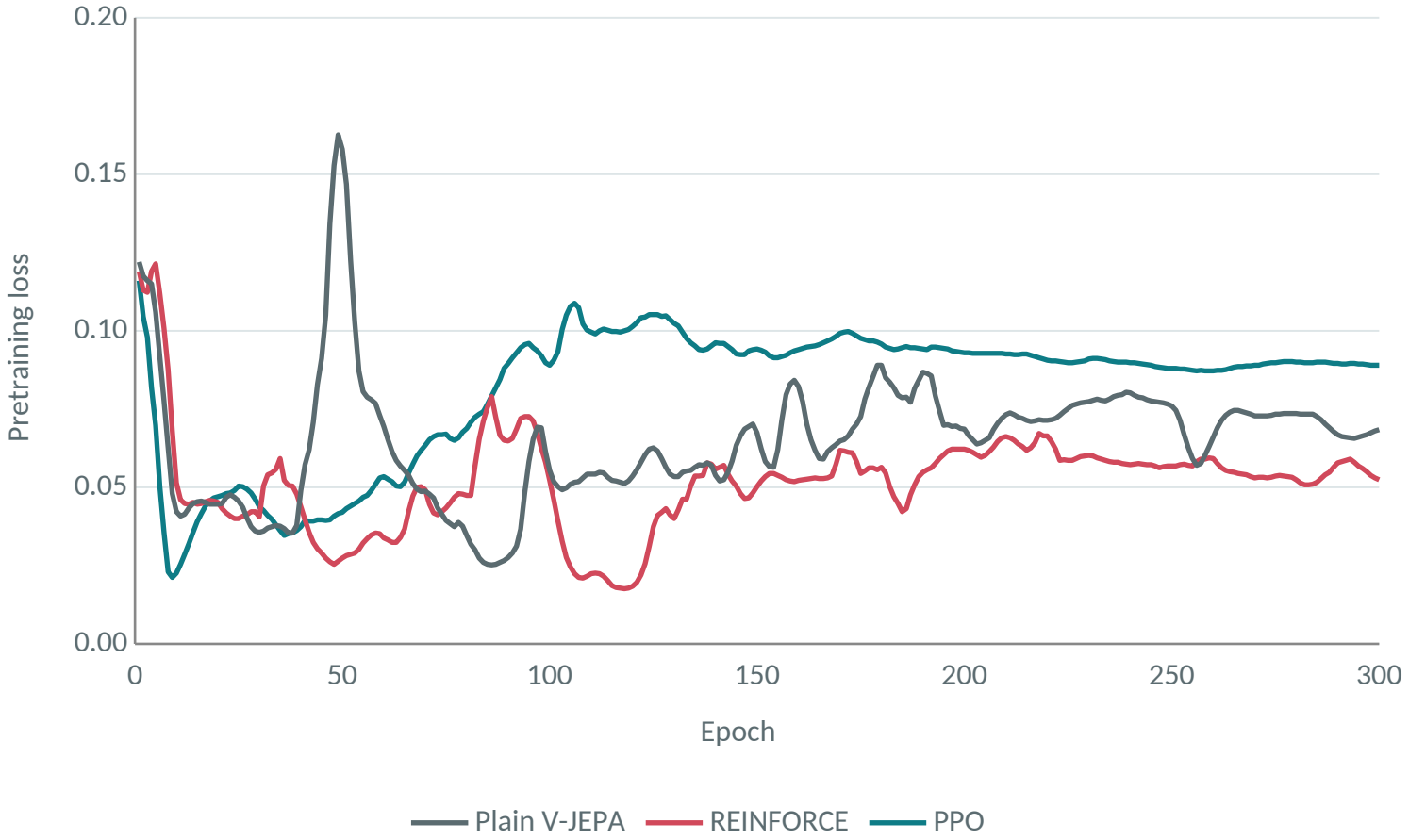
434
training videos

Run	Masking	GPU memory	Time per step
Plain V-JEPA	Random blocks	34.7 GB	1.06 s
REINFORCE	Learned block placement	45.3 GB	0.63 s
PPO	Learned token selection	17.1 GB	0.55 s

PPO keeps only 156 tokens visible, so the context encoder does much less work.

Single runs on a shared cluster at different times: memory is reliable, timings are indicative only.

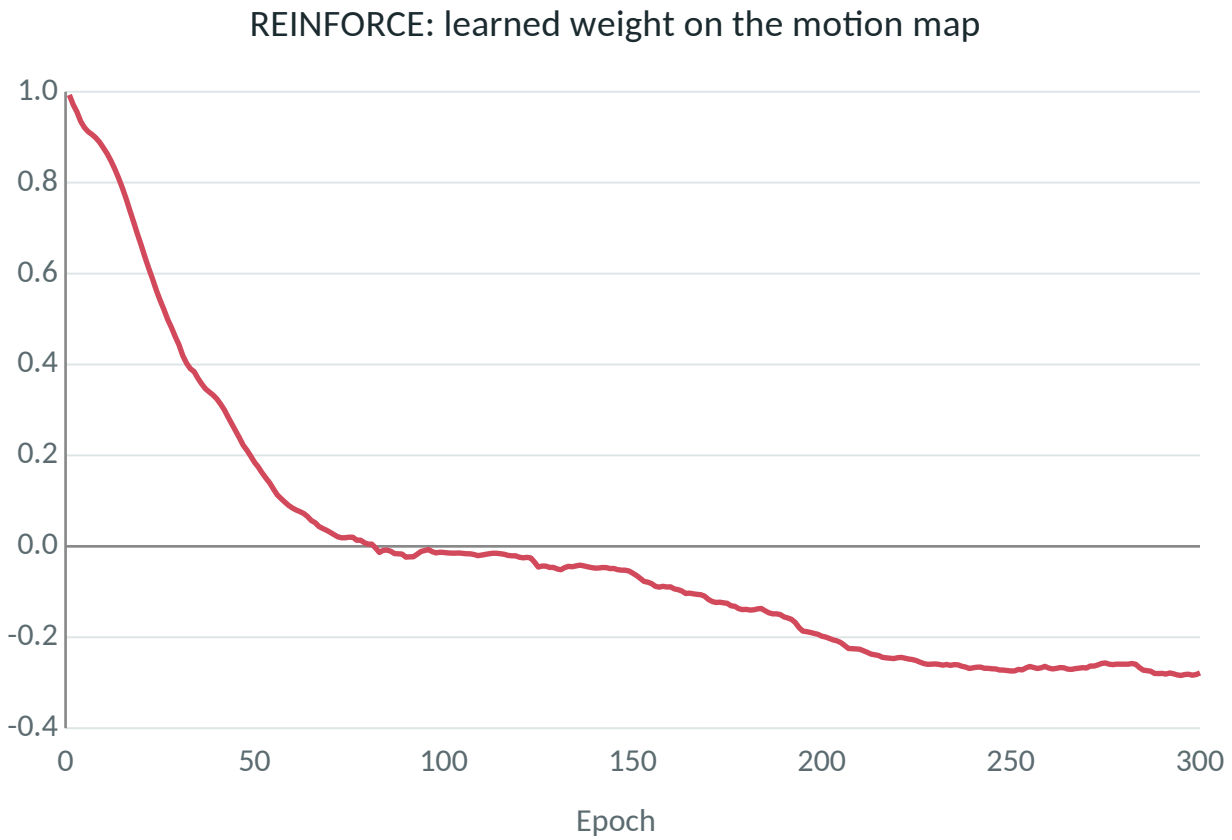
All runs show the same loss pattern: a drop, then a rise



- Loss falls to about 0.02 to 0.04 early, then climbs and levels off
- Mean of the last 20 epochs: plain 0.068, REINFORCE 0.055, PPO 0.089
- The prediction target keeps changing, so a rising loss does not mean worse features
- Runs with different masks cannot be ranked by loss

Average loss per epoch, 5-epoch moving average. PPO epochs 1 to 10 use random token masks.

Both policies stayed close to random masking



-0.28

REINFORCE motion weight at epoch 300 (started at 1.0): it learned to avoid moving regions

7.18 of 7.36

REINFORCE entropy at epoch 300 against the maximum, which means uniform random

6.48 of 7.36

PPO entropy at epoch 300: a mild preference, still close to uniform

A positive weight sends masks toward motion; a negative weight sends them away.

4

Classification

Testing the frozen encoders on the standard-plane task

A linear classifier on frozen features labels every frame

1. **Frozen encoder** pretrained weights are never updated
2. **16-frame windows** slide along each video, 8 frames apart
3. **One feature per 2 frames** average of the 196 patch tokens
4. **Linear classifier** trained on the training videos
5. **Select, then report** settings chosen on validation, scored on test

31,448

training samples, each with its video's label

2,870

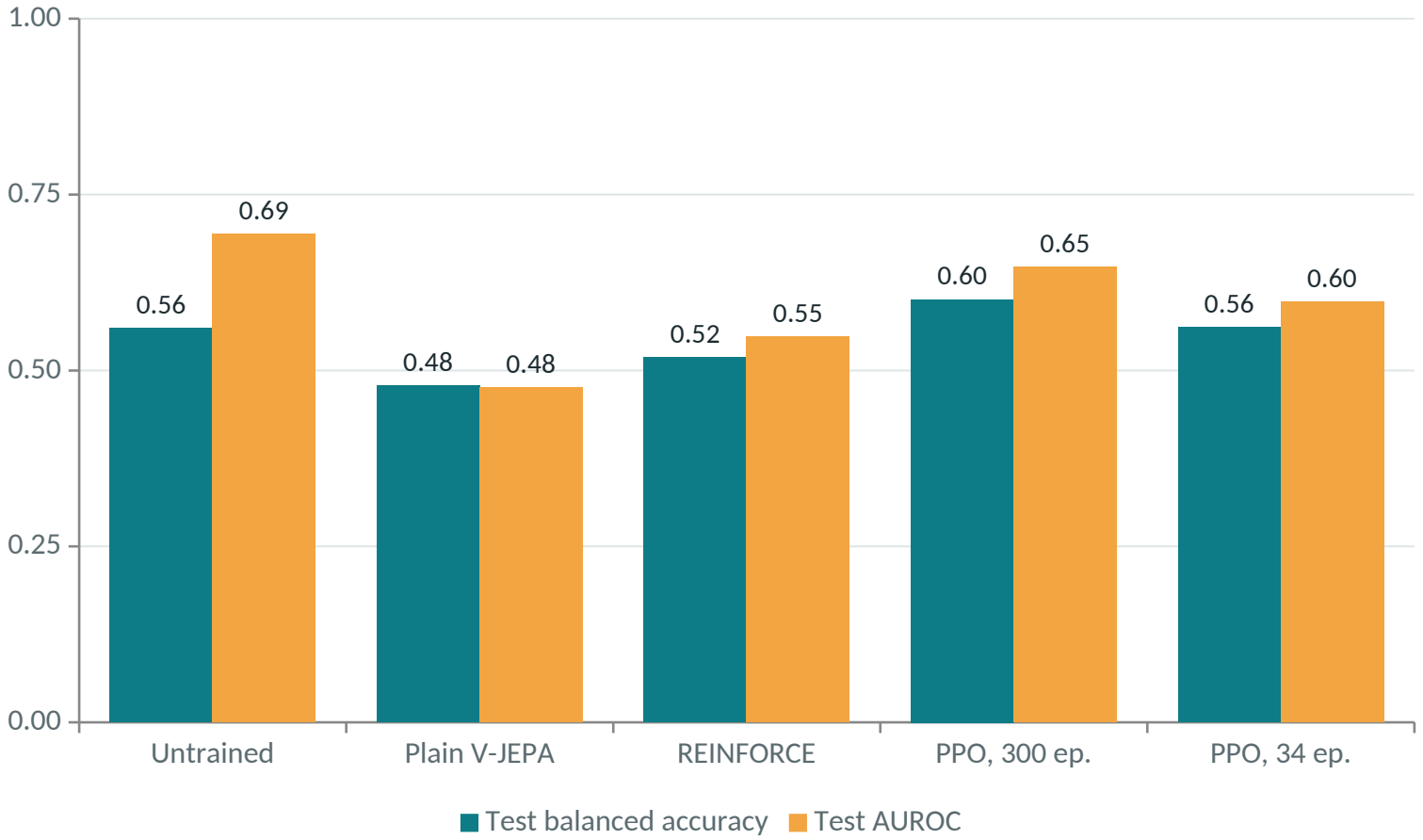
validation frames, each with its own label

8,665

test frames, each with its own label

The test set is never used to choose anything. Metrics: accuracy, balanced accuracy, F1 and AUROC.

No pretrained model clearly beats an untrained encoder



Frame-level results on the test set (8,665 frames). Chance level is 0.50 for both metrics.

- PPO at 300 epochs has the best balanced accuracy (0.60)
- The untrained encoder has the best AUROC (0.69)
- Plain V-JEPA and REINFORCE are close to chance
- All differences are small next to the distance from a useful classifier

Full results: validation and test for every model

Model	Val balanced acc.	Val AUROC	Test accuracy	Test balanced acc.	Test F1	Test AUROC
Untrained encoder	0.710	**0.834**	0.605	0.560	**0.729**	**0.694**
Plain V-JEPA, 300 epochs	0.634	0.692	0.525	0.479	0.675	0.476
REINFORCE, 300 epochs	0.586	0.592	0.484	0.519	0.315	0.548
PPO, 300 epochs	0.713	0.808	**0.617**	**0.601**	0.681	0.647
PPO, 34 epochs	**0.720**	0.795	0.586	0.561	0.680	0.598

Best value in each column is in teal. Always predicting the majority class gives 0.556 test accuracy. Classifier settings were chosen by validation balanced accuracy.

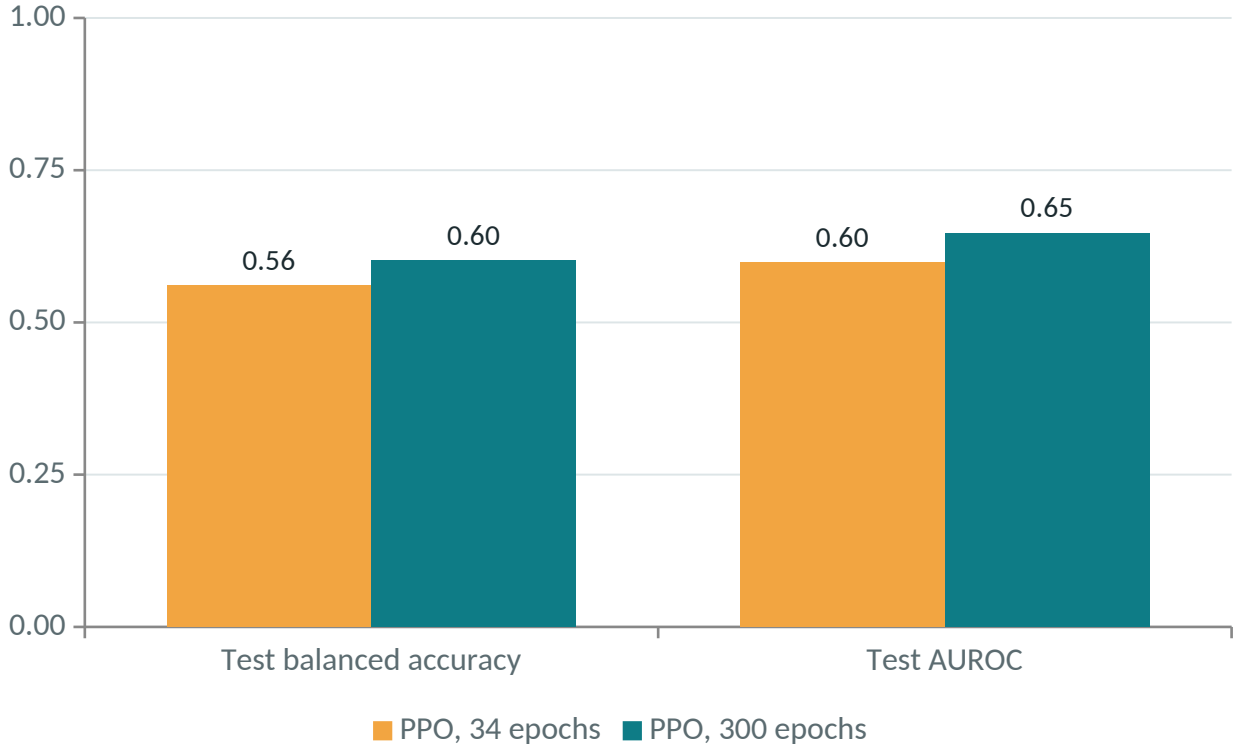
Stopping at the lowest-loss epoch did not give better features

Epoch 34

had the lowest loss (0.032) in the 300-epoch PPO run once the policy was active, so PPO was retrained to stop there.

Epoch 9

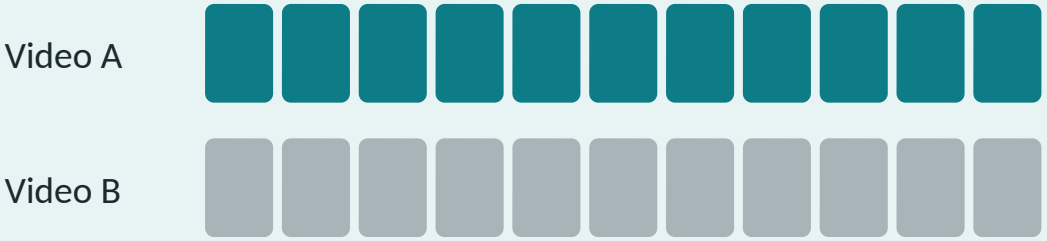
had the lowest loss (0.027) in that rerun, and epoch 34 reached 0.042. The minimum moves from run to run.



The 34-epoch model is slightly worse on test. Pretraining loss is not a reliable way to choose a checkpoint here.

Training and test ask the classifier different questions

Training



SP and NSP frames come from different videos. A classifier can separate them by how a recording looks, without learning what a standard plane is.

Validation and test



SP and NSP frames sit in the same video, a few frames apart. The classifier has to detect the moment the view changes.

SP frame

NSP frame

Also likely to hurt: averaging patch tokens discards spatial detail, and the probe reads consecutive frames while pretraining used every 4th frame.

What we can and cannot conclude

1. **The pipeline works end to end**
   Learned masking trains alongside V-JEPA in both versions, and PPO is stable after the fixes

2. **Neither policy learned a clear strategy**
   Both stayed near uniform; REINFORCE drifted away from motion

3. **Token masking with PPO is the cheapest run**
   About 17 GB of GPU memory, which leaves room for larger batches or several masks per clip

4. **The benefit of learned masking is still open**
   The current probe cannot separate the models from an untrained encoder

Next steps: fix the evaluation first, then strengthen the policy

Evaluation

- Use an attentive probe instead of averaging patch tokens, as in V-JEPA's own evaluation
- Report a within-video metric, such as AUROC computed per test video
- Add a random-token baseline to separate the policy from the change in mask type

Method

- Run several PPO passes per batch so that updates actually change the policy
- Commit to one reward direction: easy masks or hard masks
- Try group-based advantages (GRPO) in place of the weak critic
- Match video names exactly in the pretraining data loader